

AVL

AVL 树

AVL 树，是一种平衡的二叉搜索树。由于各种算法教材上对 AVL 的介绍十分冗长，造成了很多人对 AVL 树复杂、不实用的印象。但实际上，AVL 树的原理简单，实现也并不复杂。

性质

1. 空二叉树是一个 AVL 树
2. 如果 T 是一棵 AVL 树，那么其左右子树也是 AVL 树，并且， h 是其左右子树的高度
3. 树高为

平衡因子：右子树高度 - 左子树高度

▼ 树高的证明

设 n_h 为高度为 h 的 AVL 树所包含的最少节点数，则有

根据常系数非齐次线性差分方程的解法， n_h 是一个斐波那契数列。这里 n_h 的通项为：

斐波那契数列以指数的速度增长，对于树高 h 有：

因此 AVL 树的高度为 $O(\log n)$ ，这里的 n 为结点数。

过程

插入结点

与 BST（二叉搜索树）中类似，先进行一次失败的查找来确定插入的位置，插入节点后根据平衡因子来决定是否需要调整。

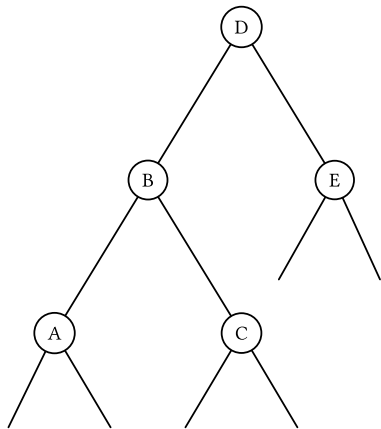
删除结点

删除和 BST 类似，将结点与后继交换后再删除。

删除会导致树高以及平衡因子变化，这时需要沿着被删除结点到根的路径来调整这种变化。

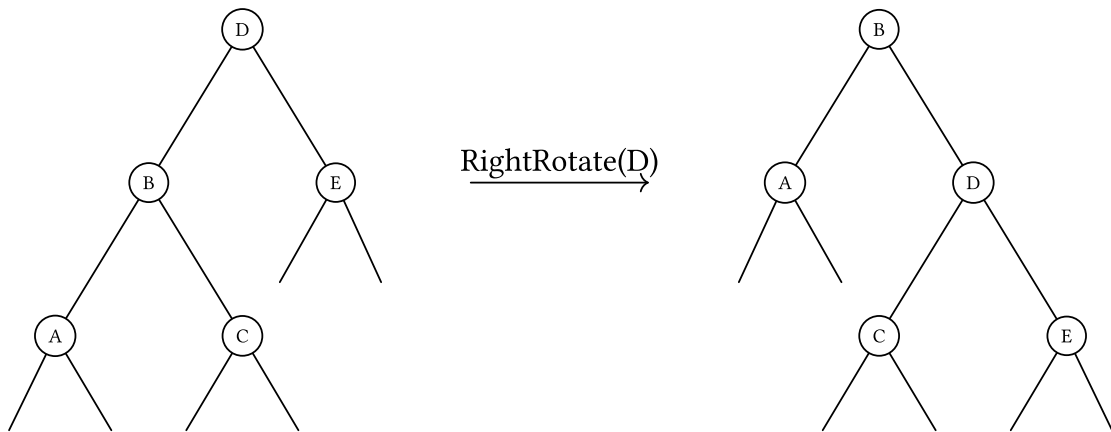
平衡的维护

插入或删除节点后，可能会造成 AVL 树的性质 2 被破坏。因此，需要沿着从被插入/删除的节点到根的路径对树进行维护。如果对于某一个节点，性质 2 不再满足，由于我们只插入/删除了一个节点，对树高的影响不超过 1，因此该节点的平衡因子的绝对值至多为 2。由于对称性，我们在此只讨论左子树的高度比右子树大 2 的情况，即下图中。此时，还需要根据 BF_L 和 BF_{LL} 的大小关系分两种情况讨论。需要注意的是，由于我们是自底向上维护平衡的，因此对节点 D 的所有后代来说，性质 2 仍然是被满足的。



设 h_A ，则有

其中 h_A 是由于节点 B 满足性质 2，因此 h_B 和 h_C 的差不会超过 1。此时我们对节点 D 进行一次右旋操作（旋转操作与其它类型的平衡二叉搜索树相同），如下图所示。

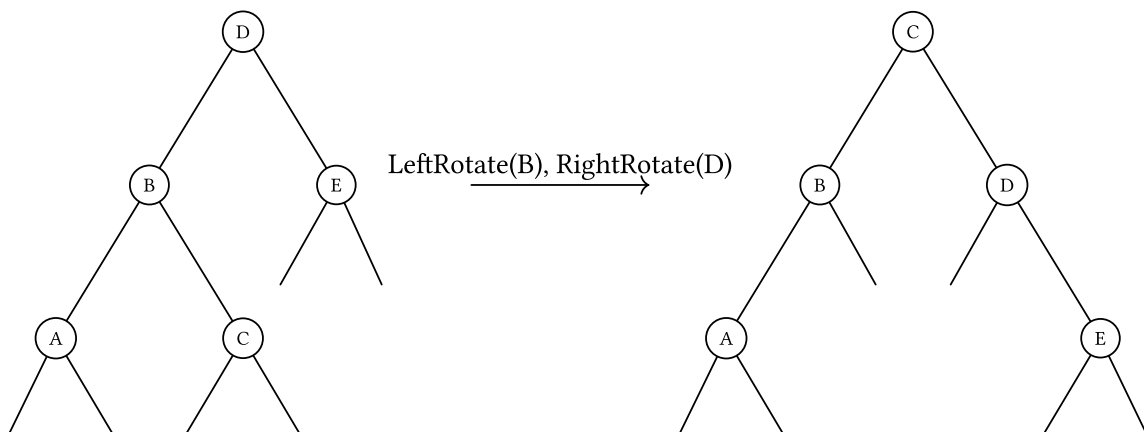


显然节点 A、C、E 的高度不发生变化，并且有

因此旋转后的节点 B 和 D 也满足性质 2。

设 h_D ，则与刚才同理，有

此时我们先对节点 B 进行一次左旋操作，再对节点 D 进行一次右旋操作，如下图所示。



显然节点 A、E 的高度不发生变化，并且 B 的新右儿子和 D 的新左儿子分别为 C 原来的左右儿子，则有

因此旋转后的节点 B、C、D 也满足性质 2。

▼ 维护平衡操作：伪代码

与其他平衡二叉搜索树相同，AVL 树中节点的高度、子树大小等信息需要在旋转时进行维护。

其他操作

AVL 树的其他操作（Predecessor、Successor、Select、Rank 等）与普通的二叉搜索树相同。

参考代码

下面的代码是用 AVL 树实现的 `Map`，即有序不可重映射：

► 参考代码

其他资料

在 [AVL Tree Visualization](#) 可以观察 AVL 树维护平衡的过程。

[维基百科 -- AVL 树](#)

本页面最近更新：2024/8/22 00:45:02, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页贡献者：[5ab-juruo](#), [ChungZH](#), [diauweb](#), [Enter-tainer](#), [GoodCoder666](#), [Great-designer](#), [iamtwz](#), [lr1d](#), [jimgreen2013](#), [mgt](#), [Rlvance](#), [Tiphereth-A](#), [Wajov](#), [Xeonacid](#)

本页面的全部内容可在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用